MAPGEN ENGINE — ULTIMATE 10-WEEK IMPLEMENTATION GAME PLAN & COMPREHENSIVE PROJECT MANUAL

***Project Target**: Procedural Map Generation Engine & Educational Visualizer Platform*

***Team Size**: 6 Members*

***Skill Baseline**: Basic Python syntax (learning Flask, Pillow, HTML5 Canvas, Procedural Algorithms, and Pytest on the job)*

***Primary Technology Stack**: Python 3.10+, NumPy, Pillow (PIL), Flask, HTML5/CSS3/Vanilla JS (Canvas API), Pytest*

SECTION 1: EXECUTIVE OVERVIEW & TEAM ROLES

The **MapGen Engine** decouples game level generation from hand-crafted static assets into seed-based mathematical synthesis. To execute this efficiently, the team is divided into **6 specialized, isolated roles** with standardized interface contracts.

Team Roles & Responsibility Matrix

Role ID & Name	Primary Ownership	Core Deliverables & Technologies	Key Interfaces
Role 1: Project Coordinator & DevOps Lead	Workspace structure, Git policies, Docker, CI/CD pipelines	Git, GitHub Actions, Docker, Shell scripts, flake8	Coordinates pull requests, release packaging, CI pipeline
Role 2: Backend & API Developer	Flask REST API server, endpoint routes, CORS, request validation	Python 3, Flask, flask-cors, REST API JSON	Exposes endpoints consumed by Role 5 UI, calls Role 3 & Role 4 modules
Role 3: Procedural Generation Developer (Algo Core)	Pure Python algorithm core (Noise, Caves, Dungeons, 1D Array Math)	Python 3 (Pure logic, zero web imports), Math	Provides deterministic 2D/1D tile arrays to Role 2 Backend
Role 4: Image & Export Developer	Image generation, Pillow PNG rendering, color palettes, JSON formatters	Python 3, Pillow (PIL), JSON serialization	Receives tile matrices from Backend, returns PNG binary streams/JSON
Role 5: Frontend UI & Canvas Developer	Single-page web app, HTML5 Canvas renderer, interactive sandbox, doc pages	HTML5, Vanilla CSS3, JavaScript (ES6+), Fetch API	Connects web UI controls to Role 2 Flask REST API
Role 6: Quality & Documentation Lead	Integration tests, code coverage, architecture guides, setup manuals	`pytest`, `pytest-cov`, Markdown documentation	Audits all roles' code via automated test suites and coverage reports

---

SECTION 2: ARCHITECTURAL GUIDELINES & INTERFACE CONTRACTS

Before writing code, all team members must understand these core engineering rules:

1. **Strict Decoupling of Algorithm Core**:

Role 3 (Algorithm Dev) must NEVER import `Flask`, `Pillow`, or web libraries inside `perlin.py`, `cellular.py`, or `bsp.py`. Algorithms must remain pure Python functions receiving parameters (seed, width, height) and returning a grid.

2. **1D Flattened Array Memory Layout**:

All 2D grids of size `W x H` are stored in memory as 1D contiguous arrays of length `W * H`.

- **2D to 1D Index Conversion**: `index = y * Width + x`
- **1D to 2D Inverse Mapping**: `x = index % Width`, `y = index // Width`

3. **Deterministic Seed Guarantee**:

Passing the same 32-bit integer `seed` to any algorithm must return the exact same tile grid output across different computers and OS environments.

4. **Standardized API Request/Response JSON Schema**:

```json
{
"seed": 42,
"algorithm": "cellular",
"width": 50,
"height": 50,
"params": { "fill_probability": 0.45, "iterations": 5 }
}
```

SECTION 3: WEEK 1 SPRINT — FAST-TRACK MINIMUM VIABLE PROTOTYPE (MVP)

***Sprint Goal**: By Day 7, build a functional, end-to-end prototype where entering a seed on a simple web page generates a 2D map on an HTML5 canvas and exports a PNG image file.*

```
[User Inputs Seed & Clicks 'Generate']
■
▼
[Flask API (/api/generate)]
■
▼
[Basic Cellular Automata / Grid Generator]
■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
▼ ▼
[JSON Matrix Response] [Pillow PNG Export]
■ ■
▼ ▼
[Rendered on HTML5 Canvas] [Saved Local PNG File]
```

Detailed Week 1 Tasks by Role

Role 1: Project Coordinator & DevOps Lead

- **Task 1.1**: Create project folder structure:
- `backend/` (Flask app and REST API)
- `backend/algorithms/` (Pure Python generator modules)
- `backend/export/` (Pillow PNG and JSON export tools)
- `frontend/` (Web UI HTML/CSS/JS)
- `tests/` (Pytest test suite)
- `documentation/` (Guides and specs)
- **Task 1.2**: Create `.gitignore` ignoring `__pycache__`, `.venv`, `.pytest_cache`, and `*.png` output files.
- **Task 1.3**: Set up Git repository and create feature branches (`feature/devops`, `feature/backend`, `feature/algo`, `feature/export`, `feature/frontend`, `feature/qa`).

Role 2: Backend & API Developer

- **Task 2.1**: Set up virtual environment and install Flask (`pip install Flask flask-cors`).
- **Task 2.2**: Write basic `backend/app.py`:
- Route `GET /health` -> returns `{"status": "ok"}`.
- Route `POST /api/generate` -> parses JSON request (`seed`, `width`, `height`), calls Role 3's prototype function, and returns JSON `{"matrix": [[...]], "seed": 42}`.

Role 3: Procedural Generation Developer (Algo Core)

- **Task 3.1**: Create `backend/algorithms/prototype_gen.py`.
- **Task 3.2**: Implement `generate_prototype_grid(seed, width=20, height=20)`:
- Seed Python's `random.seed(seed)`.
- Create a 2D matrix (`width x height`) with 45% probability of `1` (Wall) and 55% `0` (Floor).
- Apply 1 simple smoothing pass (if surrounding 8-neighbor wall count > 4, cell becomes `1`, else `0`).
- Return the 2D grid list.

Role 4: Image & Export Developer

- **Task 4.1**: Install Pillow (`pip install Pillow`).
- **Task 4.2**: Create `backend/export/prototype_export.py`.
- **Task 4.3**: Implement `export_prototype_png(matrix, filename="output_prototype.png")`:
- Map `0` -> White pixels `(255, 255, 255)` (Floor).
- Map `1` -> Black pixels `(0, 0, 0)` (Wall).
- Save as PNG image using `PIL.Image.fromarray()` or pixel drawing loop.

Role 5: Frontend UI & Canvas Developer

- **Task 5.1**: Create `frontend/index.html`:
- Number input for `Seed` (default: 42).
- Button labeled "Generate Prototype Map".
- HTML5 canvas element `<canvas id="mapCanvas" width="400" height="400"></canvas>`.
- **Task 5.2**: Create `frontend/app.js`:
- Add click event listener to "Generate" button.
- Use JavaScript `fetch()` to `POST http://localhost:5000/api/generate`.
- Loop over returned matrix and draw 20x20px black/white rectangles on the canvas context (`ctx.fillRect`).

Role 6: Quality & Documentation Lead

- **Task 6.1**: Install `pytest` (`pip install pytest`).
- **Task 6.2**: Create `tests/test_prototype.py`:
- `test_seed_reproducibility()`: Asserts running `generate_prototype_grid(42)` twice yields identical 2D grids.
- `test_health_check()`: Verifies Flask server `/health` returns HTTP code 200.
- **Task 6.3**: Write `documentation/SETUP_GUIDE.md` detailing how to install requirements and run the prototype locally.

SECTION 4: COMPREHENSIVE 10-WEEK STEP-BY-STEP IMPLEMENTATION ROADMAP

PHASE 1: FOUNDATIONS & ARCHITECTURE (WEEKS 1 - 2)

WEEK 1: Fast-Track MVP Sprint

Sprint Deliverable: Working local prototype (User inputs seed -> web canvas updates + PNG exports).

- **Role 1 (DevOps)**: Folder setup, Git repo initialization, `.gitignore`, initial branching strategy.
- **Role 2 (Backend)**: Flask setup, `/health` route, `POST /api/generate` prototype route.
- **Role 3 (Algo Core)**: `prototype_gen.py` random grid generator with basic seed smoothing.
- **Role 4 (Export Dev)**: `prototype_export.py` Pillow PNG generator script.

- **Role 5 (Frontend)**: `index.html` layout with HTML5 `<canvas>` and JavaScript `fetch()` call.
- **Role 6 (QA Lead)**: Pytest environment setup, seed determinism test, and local setup guide.

WEEK 2: Spatial Data Structures & Workspace Architecture

Weekly Objective: Transition from 2D list-of-lists to fast 1D flattened array buffers; standardize REST contracts.

- **Role 1 (DevOps)**:
- Install `flake8` linter and create `.flake8` configuration file.
- Create GitHub Actions workflow (`.github/workflows/ci.yml`) to automatically run `flake8` and `pytest` on every push.
- **Role 2 (Backend)**:
- Refactor `app.py` into modular Flask application factory.
- Install `flask-cors` and configure CORS headers to allow frontend requests from any origin.
- Standardize API error responses with HTTP status codes (400 for bad parameters, 500 for internal errors).
- **Role 3 (Algo Core)**:
- Create `backend/algorithms/spatial_grid.py`.
- Implement 1D Flattened Array class/functions:
- `get_1d_index(x, y, width)` -> returns `y * width + x`.
- `get_2d_coords(index, width)` -> returns `(index % width, index // width)`.
- Benchmark 1D memory layout vs 2D nested lists for access speed.
- **Role 4 (Export Dev)**:
- Create `backend/export/palette.py`.
- Define RGB color dictionaries for all biomes:
- `DEEP_WATER`: `(75, 85, 135)` (`#4b5587`)
- `SAND`: `(210, 190, 140)` (`#d2be8c`)
- `MEADOW`: `(145, 185, 110)` (`#91b96e`)
- `FOREST`: `(80, 135, 85)` (`#508755`)
- `CAVE_WALL`: `(26, 26, 26)` (`#1a1a1a`)
- `CAVE_FLOOR`: `(58, 58, 92)` (`#3a3a5c`)
- **Role 5 (Frontend)**:
- Create base stylesheet `frontend/styles.css`: Dark mode layout, clean typography (Inter / Roboto font), navbar, and flexbox containers.
- Add loading spinner UI component while waiting for API responses.
- **Role 6 (QA Lead)**:
- Write unit tests for 1D spatial indexing math (`tests/test_spatial_grid.py`).
- Test boundary lookups, edge coordinates `(0, 0)`, `(W-1, H-1)`, and out-of-bounds index handling.

PHASE 2: CORE PROCEDURAL ALGORITHMS DEVELOPMENT (WEEKS 3 - 5)

WEEK 3: Algorithm 1 — Perlin & Smooth Value Noise (Terrain & Overworld)

Weekly Objective: Pure Python multi-octave noise generator yielding smooth, natural terrain heightmaps.

- **Role 1 (DevOps)**:
- Configure linting rules to enforce docstrings on all math functions.
- Verify GitHub Actions CI runs noise unit tests cleanly.

- **Role 2 (Backend)**:
- Add endpoint `POST /api/generate/perlin` in `backend/app.py`.
- Parse parameters: `seed`, `width`, `height`, `scale` (float, e.g. 0.05), `octaves` (int 1..8), `persistence` (float 0.5), `lacunarity` (float 2.0).
- **Role 3 (Algo Core)**:
- Create `backend/algorithms/perlin.py` (Pure Python, zero Flask imports).
- Step 1: Implement 32-bit Bitwise Spatial Hashing:

```python
def spatial_hash(x: int, y: int, seed: int) -> float:
n = (x + y *57 + seed* 131) & 0xFFFFFFFF
n = ((n << 13) ^ n) & 0xFFFFFFFF
return 1.0 - (((n *(n* n * 15731 + 789221) + 1376312589) & 0x7FFFFFFF) / 1073741824.0)
```

- Step 2: Implement Quintic Hermite Fade Curve: `fade(t) = 6*t**5 - 15*t**4 + 10*t**3`.
- Step 3: Implement Bilinear Interpolation (`lerp(a, b, t)`).
- Step 4: Implement Multi-Octave Fractal Superposition loop.
- Step 5: Implement Biome Thresholding mapping noise values `[0.0, 1.0]` to terrain IDs (0: Deep Water, 1: Sand, 2: Meadow, 3: Forest).
- **Role 4 (Export Dev)**:
- Build `export_perlin_png()` mapping biome terrain IDs to palette colors.
- Support high-resolution PNG rendering with Pillow.
- **Role 5 (Frontend)**:
- Create `frontend/perlin.html` explanation page.
- Add interactive sliders for `Scale`, `Octaves`, `Persistence`, and `Lacunarity`.
- Draw rendered terrain heightmap onto HTML5 Canvas.
- **Role 6 (QA Lead)**:
- Write `tests/test_perlin.py`.
- Verify noise output stays strictly bounded in range `[0.0, 1.0]`.
- Test seed determinism across 100 random seed variations.

WEEK 4: Algorithm 2 — Cellular Automata (Subterranean Cave Systems)

Weekly Objective: Organic cavern generator using 4-5 rule with state hysteresis and BFS flood-fill cleanup.

- **Role 1 (DevOps)**:
- Set up automated performance benchmark in CI ensuring 20,000-tile cave generation completes under 5ms.
- **Role 2 (Backend)**:
- Add endpoint `POST /api/generate/cellular` in `backend/app.py`.
- Parse parameters: `seed`, `width`, `height`, `fill_probability` (0.45), `iterations` (5).
- **Role 3 (Algo Core)**:
- Create `backend/algorithms/cellular.py`.
- Step 1: Initialize grid with outer boundaries set to solid walls (1). Inner cells set to wall (1) or floor (0) based on `spatial_hash(x, y, seed) < fill_probability`.
- Step 2: Implement Moore (N8 8-neighbor) and Von Neumann (N4 4-neighbor) sampling kernels:

```python
```

```
def count_wall_neighbors(grid, x, y, width, height):
count = 0
for dy in (-1, 0, 1):
for dx in (-1, 0, 1):
if dx == 0 and dy == 0: continue
nx, ny = x + dx, y + dy
if nx < 0 or nx >= width or ny < 0 or ny >= height:
count += 1 # Out of bounds treated as wall
elif grid[ny * width + nx] == 1:
count += 1
return count
```

- Step 3: Implement Corrected 4-5 Rule with State Hysteresis:
- Next state = Wall (1) if WallCount > 4
- Next state = Floor (0) if WallCount < 4
- Next state = Retain Current State if WallCount == 4
- Step 4: Implement BFS Flood-Fill Connectivity Pass:
- Find all connected floor chambers.
- Identify largest connected chamber.
- Fill all smaller isolated floor pockets back into solid rock walls (1).
- **Role 4 (Export Dev)**:
- Build `export_cave_png()` using dark rock stone textures and floor highlight colors.
- **Role 5 (Frontend)**:
- Create `frontend/cellular.html` and `frontend/moore-neighborhood.html`.
- Build step-by-step animation controls allowing users to step through iteration 1, 2, 3... 5.
- **Role 6 (QA Lead)**:
- Write `tests/test_cellular.py`.
- Assert BFS flood-fill connectivity: 100% of floor tiles must be reachable from starting floor tile.

WEEK 5: Binary Space Partitioning (BSP Dungeon Architecture)

Weekly Objective: Structured room-and-corridor dungeon generator using recursive spatial tree bisection.

- **Role 1 (DevOps)**:
- Audit repo imports to guarantee `bsp.py` remains free of external dependencies.
- **Role 2 (Backend)**:
- Add endpoint `POST /api/generate/bsp` in `backend/app.py`.
- Parse parameters: `seed`, `width`, `height`, `min_room_size` (6), `max_split_depth` (4).
- **Role 3 (Algo Core)**:
- Create `backend/algorithms/bsp.py`.
- Step 1: Implement `BSPNode` data structure:

```
BSPNode(x, y, width, height, left_child=None, right_child=None, room=None)
```

- Step 2: Implement Aspect-Ratio Tree Bisection algorithm:

- If `Height / Width >= 1.25` -> Split Horizontally.
- If `Width / Height >= 1.25` -> Split Vertically.
- Else -> Pick random split axis using seed hash.
- Step 3: Implement Leaf Room Carving:
- For each leaf node, carve rectangular room floor (~70% of node dimensions) leaving wall padding.
- Step 4: Implement Centroid Corridor Routing Graph:
- Connect room centroids `(cx1, cy1)` and `(cx2, cy2)` of sibling leaf nodes using L-shaped horizontal and vertical corridors.
- **Role 4 (Export Dev)**:
- Build `export_dungeon_png()` displaying rooms (light gray), corridors (brown), and outer walls (dark slate).
- **Role 5 (Frontend)**:
- Create `frontend/bsp.html` and `frontend/bsp-tree.html`.
- Build visual tree hierarchy diagram showing how space is recursively partitioned into left/right child nodes.
- **Role 6 (QA Lead)**:
- Write `tests/test_bsp.py`.
- Verify room non-overlap, boundary constraints, and room-to-corridor connectivity graph.

PHASE 3: IMAGE RENDERING & BACKEND API SERVER (WEEKS 6 - 7)

WEEK 6: Pillow Image Export Engine & Data Serialization

Weekly Objective: Variable resolution PNG image export engine and JSON map serializer.

- **Role 1 (DevOps)**:
- Ensure Pillow C-binary dependencies compile cleanly across platform runners in GitHub Actions CI.
- **Role 2 (Backend)**:
- Add route `POST /api/export/image`: Receives tile matrix JSON -> returns binary PNG file stream (`image/png`) using Flask `send_file(io.BytesIO(...))`.
- Add route `POST /api/export/json`: Receives tile matrix JSON -> formats downloadable `.json` file attachment.
- **Role 3 (Algo Core)**:
- Optimize tile array lookup speed for export rendering pipelines.
- **Role 4 (Export Dev)**:
- Create `backend/export/image_exporter.py`:
- Support tile scale multipliers: 1x (1px per tile), 2x (2x2px per tile), 4x (4x4px per tile), 8x (8x8px per tile).
- Draw optional tile grid overlay lines.
- Add watermark legend (Seed, Algorithm Name, Dimensions).
- Create `backend/export/json_exporter.py`:
- Format map metadata (Seed, Algorithm, Width, Height, Tile Matrix array).
- **Role 5 (Frontend)**:
- Add "Export PNG" dropdown menu (1x, 2x, 4x, 8x resolution options) and "Download JSON" button to the web UI.
- **Role 6 (QA Lead)**:
- Write `tests/test_export.py`.
- Verify PNG file magic numbers (`\x89PNG\r\n\x1a\n`) and validate exported JSON against JSON Schema specifications.

WEEK 7: Production Flask REST API & Backend Hardening

Weekly Objective: Unified production API router with strict input validation and global error handling.

- **Role 1 (DevOps)**:
- Set up Waitress / Gunicorn WSGI production web server configuration for Flask backend.
- **Role 2 (Backend)**:
- Create centralized `POST /api/generate` endpoint accepting `algorithm` string ("perlin", "cellular", "bsp").
- Implement robust input validator:
- `width` and `height` must be integers between `10` and `500`.
- `seed` must be a 32-bit integer (`-2147483648` to `2147483647`).
- Return clear HTTP 400 bad request error details for invalid inputs.
- Implement request rate limiting (`Flask-Limiter`).
- **Role 3 (Algo Core)**:
- Freeze pure Python algorithm interfaces; verify 100% parameter isolation.
- **Role 4 (Export Dev)**:
- Add support for custom color theme palettes (Monochrome, Retro Arcade, Natural Earth, Cyberpunk).
- **Role 5 (Frontend)**:
- Build UI error alert notification banners for network failures or bad input parameter warnings.
- **Role 6 (QA Lead)**:
- Write comprehensive API integration test suite (`tests/test_api_integration.py`): Test invalid payloads, out-of-bounds parameters, missing fields, and rate limits.

PHASE 4: MODERN FRONTEND UI, INTERACTIVE SANDBOX & DOCS (WEEKS 8 - 9)

WEEK 8: Interactive HTML5 Canvas Playground & Control Panel

Weekly Objective: Real-time map preview playground with pan, zoom, and seed randomizer controls.

- **Role 1 (DevOps)**:
- Verify static web asset caching rules and web server directory hosting.
- **Role 2 (Backend)**:
- Benchmark API endpoint response time under rapid control slider adjustments.
- **Role 3 (Algo Core)**:
- Assist frontend developer with client-side parameter bounds and defaults.
- **Role 4 (Export Dev)**:
- Provide hex color constants to frontend dev for seamless Canvas-to-PNG visual fidelity.
- **Role 5 (Frontend)**:
- Build Interactive Live Sandbox at the bottom of `frontend/index.html`.
- Add control panel:
- Algorithm Selector (Perlin, Cellular Automata, BSP).
- Seed Input Box + "■ Random Seed" button.
- Dynamic Sliders: Scale, Octaves, Fill Probability, Room Sizes.
- Live Map Preview Canvas with drag-to-pan and scroll-to-zoom support.
- Hover Tooltip displaying: Tile X, Tile Y, Biome Type, and Raw Value.

- **Role 6 (QA Lead)**:
- Perform cross-browser verification (Chrome, Firefox, Edge, Safari, Mobile viewports).

WEEK 9: Educational Algorithm Pages & Code Visualizers

- **Role 1 (DevOps)**:
- Run HTML5 / CSS3 validation tools and check asset paths.
- **Role 2 (Backend)**:
- Ensure API doc code snippets match production routes.
- **Role 3 (Algo Core)**:
- Review mathematical explanations on doc pages for technical accuracy.
- **Role 4 (Export Dev)**:
- Render sample map preview images for documentation cards.
- **Role 5 (Frontend)**:
- Build dedicated educational pages:
- `perlin.html` (Noise math, Hermite curves, Lerp, Octave layering).
- `cellular.html` (4-5 rule, State hysteresis, BFS flood fill).
- `bsp.html` (Recursive space splitting, room carving, corridor graphs).
- `spatial-grid.html` (2D to 1D array index math).
- `moore-neighborhood.html` (Chebyshev N8 vs Manhattan N4 stencils).
- `bsp-tree.html` (Binary tree node hierarchy visualizer).
- `spatial-hash.html` (Bitwise integer spatial hashing).
- Format syntax-highlighted code blocks showing pure Python logic.
- Implement mobile responsive drawer navigation menu.
- **Role 6 (QA Lead)**:
- Audit all internal navigation links, proofread math formulas, and check responsive image layouts.

PHASE 5: TESTING, CI/CD, DOCKER & LAUNCH (WEEK 10)

WEEK 10: Full Pipeline Integration, Dockerization & Launch

Weekly Objective: 100% automated test coverage verification, Docker packaging, and final release.

- **Role 1 (DevOps)**:
- Write production `Dockerfile`:

```dockerfile
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "backend.app:app"]
```

- Write `docker-compose.yml` orchestrating Flask backend and static web UI server.
- Finalize GitHub Actions CI pipeline with status badge in `README.md`.
- **Role 2 (Backend)**:
- Perform security audit, freeze endpoint contracts, and verify `/health` monitoring endpoint.
- **Role 3 (Algo Core)**:
- Conduct final performance benchmarking report (< 5ms per 20,000-tile map).
- **Role 4 (Export Dev)**:
- Verify PIL memory cleanup and image stream resource releasing under high concurrent requests.
- **Role 5 (Frontend)**:
- Add visual polish: dark mode glassmorphism UI elements, micro-animations, hover effects, and crisp canvas rendering.
- **Role 6 (QA Lead)**:
- Execute full test suite (`pytest --cov=backend`).
- Verify test coverage threshold meets or exceeds **85%**.
- Compile final `documentation/PROJECT_EXPLANATION.txt` and user user setup manual.

SECTION 5: ALGORITHMIC DEEP-DIVES FOR BEGINNER DEVELOPERS

1. Perlin & Smooth Value Noise Math

Noise generation avoids blocky grid randomness by smoothing values across spatial grid coordinates:

1. **Bitwise Spatial Hashing**: Maps integer `(x, y, seed)` to a deterministic float `[-1.0, 1.0]`.
2. **Quintic Hermite Fade Curve**: Smooths coordinate fractions $t \in [0, 1]$ using $fade(t) = 6t^5 - 15t^4 + 10t^3$ (ensures continuous derivatives C^2).
3. **Bilinear Interpolation (Lerp)**: Blends 4 lattice corner values: $lerp(a, b, t) = a + t \times (b - a)$.
4. **Multi-Octave Fractal Superposition**: Sums multiple noise passes at increasing frequencies (2^o) and decreasing amplitudes ($persistence^o$).

2. Cellular Automata Cave Generation

Simulates cave formation over discrete iterations:

1. **Initial Fill**: Seed hash fills grid cells with 45% initial wall probability.
2. **4-5 Rule with State Hysteresis**:
- If 8-neighbor Wall Count > 4 $\rightarrow$ Cell becomes Wall (1).
- If 8-neighbor Wall Count < 4 $\rightarrow$ Cell becomes Floor (0).
- If 8-neighbor Wall Count == 4 $\rightarrow$ Cell retains current state (prevents erosion oscillations).
3. **BFS Flood Fill**: Queue-based traversal identifies isolated unplayable pockets and fills them back into solid walls.

3. Binary Space Partitioning (BSP) Dungeon Generation

Generates architectural dungeon layouts:

1. **Aspect-Ratio Tree Bisection**: Nodes split vertically if $W/H \ge 1.25$, horizontally if $H/W \ge 1.25$, or randomly.
2. **Leaf Room Carving**: Rectangular room floors carved inside leaf node bounds (~70% dimension ratio).
3. **Centroid Corridor Routing**: Connects room centroids $(cx_1, cy1)$ to (cx_2, cy_2) via L-shaped horizontal/vertical corridor lines.

SECTION 6: RECOMMENDED LEARNING RESOURCES

Python & General Programming

- ■ Python 3 Official Documentation
- ■ Real Python: Python Basics & Virtual Environments

Procedural Algorithms & Mathematics

- ■ Understanding Perlin Noise (Adrian Biagioli)
- ■■ RogueBasin: Cellular Automata Caverns
- ■ RogueBasin: BSP Dungeon Generation
- ■ Sebastian Lague: Procedural Landmass & Cave Generation (YouTube)

Web & API Frameworks

- ■ Flask Official Quickstart Guide
- ■■ Pillow (PIL) Image Processing Guide
- ■ MDN Web Docs: HTML5 Canvas API Tutorial
- ■ MDN Web Docs: Fetch API Tutorial

Testing & DevOps

- ■ Real Python: Effective Testing With Pytest
- ■ Docker Official Getting Started Guide